PLSQL EXERCISE 1 AND 3

```
SQL> CREATE TABLE Customers (
  2      CustomerID NUMBER PRIMARY KEY,
  3      Name VARCHAR2(100),
  4      DOB DATE,
  5      Balance NUMBER,
  6      LastModified DATE
  7  );

Table created.

SQL> CREATE TABLE Accounts (
  2      AccountID NUMBER PRIMARY KEY,
  3      CustomerID NUMBER,
  4      AccountType VARCHAR2(20),
  5      Balance NUMBER,
  6      LastModified DATE,
  7      FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
  8  );

Table created.

SQL>
SQL> CREATE TABLE Transactions (
  2      TransactionID NUMBER PRIMARY KEY,
  3      AccountID NUMBER,
  4      TransactionDate DATE,
  5      Amount NUMBER,
  6      TransactionType VARCHAR2(10),
  7      FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)
  8  );

Table created.

SQL>
SQL> CREATE TABLE Loans (
  2      LoanID NUMBER PRIMARY KEY,
  3      CustomerID NUMBER,
  4      LoanAmount NUMBER,
  5      InterestRate NUMBER,
  6      StartDate DATE,
  7      EndDate DATE,
  8      FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
  9  );

Table created.

SQL>
SQL> CREATE TABLE Employees (
  2      EmployeeID NUMBER PRIMARY KEY,
  3      Name VARCHAR2(100),
  4      Position VARCHAR2(50),
  5      Salary NUMBER,
  6      Department VARCHAR2(50),
  7      HireDate DATE
  8  );

Table created.
```

```
SQL> select *from Customers;

CUSTOMERID
----------
NAME
--------------------------------------------------------------------------------
DOB          BALANCE LASTMODIF
--------- ---------- ---------
         1
John Doe
15-MAY-50      15000 18-JUN-26

         2
Jane Smith
20-JUL-90       8000 18-JUN-26

CUSTOMERID
----------
NAME
--------------------------------------------------------------------------------
DOB          BALANCE LASTMODIF
--------- ---------- ---------


SQL> select *from Accounts;

 ACCOUNTID CUSTOMERID ACCOUNTTYPE            BALANCE LASTMODIF
---------- ---------- -------------------- ---------- ---------
         1          1 Savings                    1000 18-JUN-26
         2          2 Checking                   1500 18-JUN-26

SQL> select *from Loans;

    LOANID CUSTOMERID LOANAMOUNT INTERESTRATE STARTDATE ENDDATE
---------- ---------- ---------- ------------ --------- ---------
         1          1       5000            4 18-JUN-26 08-JUL-26
         2          2      10000            7 18-JUN-26 17-AUG-26

SQL>
```

```
SQL> select *from Employees;

EMPLOYEEID
----------
NAME
--------------------------------------------------------------------------------
POSITION                                             SALARY
-------------------------------------------------- ----------
DEPARTMENT                                         HIREDATE
-------------------------------------------------- ---------
         1
Alice Johnson
Manager                                                70000
HR                                                 15-JUN-15


EMPLOYEEID
----------
NAME
--------------------------------------------------------------------------------
POSITION                                             SALARY
-------------------------------------------------- ----------
DEPARTMENT                                         HIREDATE
-------------------------------------------------- ---------
         2
Bob Brown
Developer                                              60000
IT                                                 20-MAR-17
```

Exercise 1 – Scenario 1(Apply 1% discount to customers above 60 years)

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2      v_age NUMBER;
  3  BEGIN
  4      FOR cust IN (
  5          SELECT c.CustomerID,
  6                 c.DOB,
  7                 l.LoanID
  8          FROM Customers c
  9          JOIN Loans l
 10          ON c.CustomerID = l.CustomerID
 11      )
 12      LOOP
 13
 14          v_age := FLOOR(MONTHS_BETWEEN(SYSDATE,cust.DOB)/12);
 15
 16          IF v_age > 60 THEN
 17
 18              UPDATE Loans
 19              SET InterestRate = InterestRate - 1
 20              WHERE LoanID = cust.LoanID;
 21
 22              DBMS_OUTPUT.PUT_LINE(
 23              '1% discount applied for Customer ID '
 24              || cust.CustomerID);
 25
 26          END IF;
 27
 28      END LOOP;
 29
 30      COMMIT;
 31  END;
 32  /
1% discount applied for Customer ID 1

PL/SQL procedure successfully completed.
```

Exercise 1 - Scenario 2(Promote customers with balance greater than

10000)

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> BEGIN
  2      FOR cust IN (
  3          SELECT CustomerID, Balance
  4          FROM Customers
  5      )
  6      LOOP
  7
  8          IF cust.Balance > 10000 THEN
  9
 10              DBMS_OUTPUT.PUT_LINE(
 11              'Customer ID '
 12              || cust.CustomerID
 13              || ' has been promoted to VIP.');
 14
 15          END IF;
 16
 17      END LOOP;
 18  END;
 19  /
Customer ID 1 has been promoted to VIP.

PL/SQL procedure successfully completed.
```

Exercise 1 - Scenario 3(Send reminders for loans due within next 30 days)

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> BEGIN
  2      FOR loan_rec IN (
  3          SELECT c.Name,
  4                 l.EndDate
  5          FROM Customers c
  6          JOIN Loans l
  7          ON c.CustomerID = l.CustomerID
  8          WHERE l.EndDate BETWEEN SYSDATE
  9                              AND SYSDATE+30
 10      )
 11      LOOP
 12
 13          DBMS_OUTPUT.PUT_LINE(
 14          'Reminder: Loan for '
 15          || loan_rec.Name
 16          || ' is due on '
 17          || TO_CHAR(loan_rec.EndDate,'DD-MON-YYYY'));
 18
 19      END LOOP;
 20  END;
 21  /
Reminder: Loan for John Doe is due on 08-JUL-2026

PL/SQL procedure successfully completed.
```

Exercise 3 : Scenario 1: Process Monthly Interest

```
SQL> CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
  2  IS
  3  BEGIN
  4      UPDATE Accounts
  5      SET Balance = Balance + (Balance * 0.01)
  6      WHERE AccountType = 'Savings';
  7
  8      COMMIT;
  9
 10      DBMS_OUTPUT.PUT_LINE(
 11          'Monthly interest has been applied successfully.'
 12      );
 13  END;
 14  /

Procedure created.

SQL> EXEC ProcessMonthlyInterest;
Monthly interest has been applied successfully.

PL/SQL procedure successfully completed.

SQL>
```

Exercise 3: Scenario 2: Update Employee Bonus

```
SQL> CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
  2      p_department IN VARCHAR2,
  3      p_bonus_percent IN NUMBER
  4  )
  5  IS
  6  BEGIN
  7      UPDATE Employees
  8      SET Salary = Salary + (Salary * p_bonus_percent / 100)
  9      WHERE Department = p_department;
 10
 11      COMMIT;
 12
 13      DBMS_OUTPUT.PUT_LINE(
 14          'Bonus updated successfully for '
 15          || p_department
 16          || ' department.'
 17      );
 18  END;
 19  /

Procedure created.

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> EXEC UpdateEmployeeBonus('IT',10);
Bonus updated successfully for IT department.

PL/SQL procedure successfully completed.

SQL>
```

Exercise 3: Scenario 3: Transfer Funds Between Accounts

```
SQL> CREATE OR REPLACE PROCEDURE TransferFunds(
  2      p_source_account IN NUMBER,
  3      p_target_account IN NUMBER,
  4      p_amount IN NUMBER
  5  )
  6  IS
  7      v_balance NUMBER;
  8  BEGIN
  9
 10      SELECT Balance
 11      INTO v_balance
 12      FROM Accounts
 13      WHERE AccountID = p_source_account;
 14
 15      IF v_balance >= p_amount THEN
 16
 17          UPDATE Accounts
 18          SET Balance = Balance - p_amount
 19          WHERE AccountID = p_source_account;
 20
 21          UPDATE Accounts
 22          SET Balance = Balance + p_amount
 23          WHERE AccountID = p_target_account;
 24
 25          COMMIT;
 26
 27          DBMS_OUTPUT.PUT_LINE(
 28              'Amount transferred successfully.'
 29          );
 30
 31      ELSE
 32
 33          DBMS_OUTPUT.PUT_LINE(
 34              'Insufficient balance. Transfer failed.'
 35          );
 36
 37      END IF;
 38
 39  EXCEPTION
```

```
 37      END IF;
 38
 39  EXCEPTION
 40      WHEN NO_DATA_FOUND THEN
 41          DBMS_OUTPUT.PUT_LINE(
 42              'Account not found.'
 43          );
 44
 45      WHEN OTHERS THEN
 46          ROLLBACK;
 47          DBMS_OUTPUT.PUT_LINE(
 48              'An error occurred during transfer.'
 49          );
 50  END;
 51  /

Procedure created.

SQL> SELECT AccountID, Balance
  2  FROM Accounts;

 ACCOUNTID    BALANCE
---------- ----------
         1       1010
         2       1500

SQL>
```